

Contexto

Um detalhe que você já percebeu usando streaming: a plataforma não toca só música. Tem podcast, tem audiolivro, tem vários tipos de conteúdo convivendo no mesmo app. No fundo, uma música e um podcast têm muita coisa em comum: os dois têm título, duração, um id, e os dois podem ser reproduzidos.

Nesta fase o Sonora vai reconhecer mais de um tipo de conteúdo, e você vai modelar isso com herança. A ideia é ter uma superclasse Conteudo, que concentra o que é comum a qualquer conteúdo, e duas subclasses que a especializam: Musica e Podcast. Assim você reaproveita código e deixa cada subclasse cuidar só do que a torna diferente.

Preparação

Como nas fases anteriores, esta atividade é incremental. Clone o projeto da fase passada e trabalhe por cima dele:

- **Copie** sonora-fase04 **para uma nova pasta** sonora-fase05.
- Mantenha o pacote sonora e tudo que já funcionava.
- Nada do que você já entregou pode quebrar: ao terminar a Fase 05, o projeto continua compilando e rodando.

O que você vai praticar

- Herança com extends e a relação "é um(a)".
- Superclasse e subclasse, e a leitura da hierarquia de herança.
- Membros protected e o símbolo # na UML.
- Sobrescrita de método com @Override.
- Reuso do método da superclasse com super.metodo().
- Encadeamento de construtores com super(...).
- A classe Object e a sobrescrita de toString().

Antes de escrever qualquer código, atualize o diagrama de classes com a nova hierarquia: Musica e Podcast herdando de Conteudo, com a seta de especialização, os membros protegidos marcados com # e os atributos de cada classe no lugar certo. Use esse diagrama como plano para a implementação e **envie junto com os códigos**.

Parte A: crie a superclasse Conteudo

Comece pela base da hierarquia. A classe Conteudo guarda o que todo conteúdo tem, independentemente de ser música ou podcast.

Requisitos da Conteudo:

- Um id próprio, gerado por um contador estático, no mesmo padrão das outras classes.
- título (texto, obrigatório, não pode ser nulo nem vazio).
- duracaoSegundos (inteiro, deve ser maior que zero).

- Getters e setters, sempre validando as regras acima e lançando `IllegalArgumentException` quando violadas, como você já faz desde a Fase 02.
- Um método `reproduzir()` que imprime na tela que o conteúdo está sendo reproduzido.
- O acesso ao `id` (o setter e/ou o contador) deve ser `protected`, para que as subclasses consigam usar, mas o restante do sistema não.

Esqueleto sugerido (preencha os corpos):

```
public class Conteudo {

    private static int contador = 0;
    private int id;
    private String titulo;
    private int duracaoSegundos;

    public Conteudo(String titulo, int duracaoSegundos) {
        this.id = ++contador;
        setTitulo(titulo);
        setDuracaoSegundos(duracaoSegundos);
    }

    public int getId() {
        return id;
    }

    protected void setId(int id) {
        this.id = id;
    }

    // getTitulo, setTitulo (valida vazio/nulo),
    // getDuracaoSegundos, setDuracaoSegundos (valida > 0)

    public void reproduzir() {
        System.out.println("Reproduzindo: " + toString());
    }

    @Override
    public String toString() {
        return "[" + getId() + "] " + titulo
            + " (" + duracaoSegundos + "s)";
    }

}
```

Parte B: especialize com Musica e Podcast

Agora crie as subclasses. Cada uma herda de `Conteudo` e acrescenta apenas o que é específico dela.

1. Faça `Musica` `extends Conteudo`, adicionando `artista` e `album`. Se a `Musica` já existia das fases anteriores, faça-a passar a herdar de `Conteudo` e deixe nela só o que é específico (o `id`, o `título` e a `duração` agora vêm da superclasse).
2. Crie `Podcast` `extends Conteudo`, adicionando `apresentador` e `numeroEpisodio` (inteiro, deve ser maior ou igual a 1).

3. Nos construtores das subclasses, chame `super(...)` como primeiro comando para inicializar a parte herdada (título e duração) e deixe cada subclasse cuidar só dos seus próprios campos.
4. Mantenha as validações comuns (título, duração) em `Conteudo` e as específicas (episódio maior ou igual a 1) em `Podcast`.

Exemplo do construtor da subclasse:

```
public class Musica extends Conteudo {  
  
    private String artista;  
    private String album;  
  
    public Musica(String titulo, int duracaoSegundos,  
                  String artista, String album) {  
        super(titulo, duracaoSegundos);  
        setArtista(artista);  
        setAlbum(album);  
    }  
  
    // getters e setters proprios  
    // + toString() sobrescrito (ver Parte C)  
}
```

Parte C: sobrescrita, super e a classe Object

Toda classe em Java herda de `Object`, e é de lá que vem o `toString()`. Você vai sobrescrevê-lo para que cada tipo de conteúdo se apresente do seu próprio jeito, reaproveitando a parte comum.

1. Em `Conteudo`, o `toString()` já devolve a parte comum (id, título e duração), como no esqueleto da Parte A.
2. Sobrescreva `toString()` em `Musica` e em `Podcast`. Em cada uma, chame `super.toString()` para reaproveitar a parte comum e acrescente o que é específico. Por exemplo, `Musica` acrescenta artista e álbum; `Podcast` acrescenta o número do episódio e o apresentador.
3. Use `@Override` em todo método sobrescrito. Sem a anotação, um erro de digitação na assinatura transforma sua sobrescrita em uma sobrecarga silenciosa, e o compilador não te avisa.

Exemplo de sobrescrita reaproveitando a superclasse:

```
@Override  
public String toString() {  
    return super.toString() + " - " + artista  
        + " (" + album + ")";  
}
```

Parte D: demonstre no App

No App (método `main`), mostre a hierarquia funcionando:

1. Crie pelo menos duas músicas e um podcast.
2. Chame `reproduzir()` em cada um. Repare que o método está na superclasse e é herdado pelas duas subclasses, sem precisar reescrever.



3. Imprima cada objeto (o `toString()` sobrescrito deve mostrar os dados específicos de cada tipo).
4. Mantenha o tratamento de exceções com `try/catch` como nas fases anteriores, de modo que o menu nunca quebre.

Restrições desta fase

- **Herança simples apenas.** Sem interfaces: elas ainda não foram vistas.
- **Sem classes ou métodos abstratos e sem final.** Esses recursos são da segunda parte de herança e serão cobrados depois. Aqui a superclasse `Conteudo` é uma classe concreta comum.
- **Sem instanceof, sem cast de tipos** e sem tratar coleções da superclasse de forma polimórfica: isso é assunto da fase de polimorfismo. A `Playlist` e a `Plataforma` continuam como estão.
- **Coleções:** apenas `ArrayList` onde já era usado (Fase 04). Nada de `LinkedList`, `Map`, `Set` ou similares.
- **Sem entrada/saída de arquivo** e sem banco de dados.
- **Mantenha o tratamento de exceções** da Fase 02 em todo o código novo.